

ShipPulse

Solution Guide

Exact answers, why, and troubleshooting

Org: AzureDevOpsC07AA | Region: Central India | Date: March 2, 2026

1. Final target state

Each engineer has a separate repo in the same org, a separate set of Azure resources, and a complete startup-style flow:

- VM in dev with Nginx
- Logic App that starts the stopped VM
- Backend in Azure App Service
- Frontend in Azure Static Web Apps
- CI + multi-environment deployment in GitHub Actions
- Key Vault + Managed Identity
- Application Insights + one alert

2. Repo model and why

Engineer	Repo	Suffix
Engineer 1	AzureDevOpsC07AA/shippulse-eng1	e1c07aa
Engineer 2	AzureDevOpsC07AA/shippulse-eng2	e2c07aa

Why separate repos? It prevents secret collisions, Azure naming collisions, and branch conflicts. It also makes each engineer fully responsible for their own CI/CD and Azure resources.

3. What is already coded for them

The starter repo is not empty. It already contains real code.

- Backend endpoint: `GET /health`
- Backend endpoint: `GET /api/feedback`
- Backend endpoint: `POST /api/feedback`
- Frontend page that submits and displays feedback
- Bicep modules for core Azure resources
- GitHub Actions workflow files

This is intentionally closer to the KodeKloud style: do not waste student time on scaffolding when the learning goal is Azure and DevOps.

4. Day 2 answer - VM + Nginx + Bicep

Exact sequence:

```
az group create -n rg-shippulse-dev -l centralindia
az group create -n rg-shippulse-prod -l centralindia
```

Then in Cloud Shell:

```
ssh-keygen -t rsa -b 4096 -f ~/.ssh/shippulse_id_rsa -N ""
cat ~/.ssh/shippulse_id_rsa.pub
```

Paste the public key into `infra/env/dev.bicepparam`, set `sshAllowedCidr`, and replace `demo01` with the assigned suffix.

```
az deployment group validate -g rg-shippulse-dev -f infra/main.bicep -p
infra/env/dev.bicepparam
az deployment group what-if -g rg-shippulse-dev -f infra/main.bicep -p
infra/env/dev.bicepparam
az deployment group create -g rg-shippulse-dev -f infra/main.bicep -p
infra/env/dev.bicepparam
```

SSH and install Nginx:

```
ssh -i <PRIVATE_KEY_PATH> azureuser@<VM_PUBLIC_IP>
sudo apt-get update
sudo apt-get install -y nginx
sudo systemctl enable --now nginx
```

Why this is included: you already taught VM basics, port 80, IP allowlisting, start/stop, and Nginx. This lab reuses that knowledge instead of skipping it.

Common issues

- SSH denied: wrong private key or `sshAllowedCidr` not updated to your current public IP/32.
- Port 80 unreachable: NSG rule missing or VM not running.
- Key Vault deploy fails: Key Vault name not globally unique or contains invalid characters.

5. Day 3 answer - Backend deployment to App Service

Use GitHub environment secrets. Beginner-friendly path: start with Service Principal secret-based auth because the engineers already learned it.

Required dev secrets:

- AZURE_CLIENT_ID
- AZURE_TENANT_ID
- AZURE_SUBSCRIPTION_ID
- AZURE_RG_DEV
- AZURE_WEBAPP_NAME_DEV

Why App Service? It is the right complexity level for a small .NET startup API. It is easier to operate than AKS and integrates well with GitHub Actions, Key Vault, and Application Insights.

Common issues

- `azure/login` fails: wrong tenant/client/subscription values or missing secret.
- Deploy succeeds but site errors: confirm App Service is Linux + .NET 8 and open `/health` first.
- Workflow uses wrong app name: check the suffix and the GitHub environment secret value.

6. Day 4 answer - Frontend deployment to Static Web Apps

Use Portal for Static Web Apps creation because it is faster and more beginner-safe. Portal also makes it easier to get the deployment token.

Required secrets:

- AZURE_STATIC_WEB_APPS_API_TOKEN_DEV
- AZURE_STATIC_WEB_APPS_API_TOKEN_PROD
- API_BASE_URL_DEV
- API_BASE_URL_PROD

Why Blazor WebAssembly? It is a real .NET frontend and builds with dotnet only. That matches the office laptop restriction and keeps the full stack inside .NET.

Common issues

- Frontend loads but API calls fail: API base URL secret is wrong.
- CORS errors: backend is reachable but origin is not allowed. For the assignment, use the provided fallback and note tighter CORS as a next step.
- Deep links fail in SWA: ensure `staticwebapp.config.json` exists in `wwwroot`.

7. Day 5 answer - Key Vault and Managed Identity

Create the secret `ExternalApi--ApiKey` in each Key Vault.

Assign `Key Vault Secrets User` to the Web App managed identity.

Why this matters: deployment identity and runtime identity should be separated.

- Deployment identity = GitHub Actions authenticates to Azure and deploys resources.
- Runtime identity = Web App managed identity reads secrets from Key Vault.

Common issues

- Key Vault reference unresolved: RBAC assignment missing or not propagated yet. Wait a few minutes and restart the app.
- Secret name mismatch: must exactly match `ExternalApi--ApiKey`.

8. Day 6 answer - Logic App to start stopped VM

This directly matches what you already taught: Logic App can be used to start a stopped VM.

Recommended student flow:

- Create Logic App Consumption in `rg-shippulse-dev` named `la-start-vm-dev-<suffix>`
- Trigger: Recurrence
- Time zone: India Standard Time
- Testing interval: every 5 minutes
- Action: Start virtual machine `vm-shippulse-jump-<suffix>` in `rg-shippulse-dev`

After successful test, change recurrence to weekdays 9:00 AM IST.

Why this is realistic: startups often power off non-production VMs to save money and then restart them automatically.

Common issues

- Logic App action cannot find the VM: wrong subscription, RG, or VM name.
- Logic App does not start VM: connector/connection not authorized correctly.
- Schedule confusion: wrong time zone selected. Must be India Standard Time.

9. Day 6 answer - Monitoring and alerts

Use Application Insights and one basic alert on the prod Web App.

Why this is production-grade: if the app fails in production, the team needs a signal.

For a one-week lab, you do not need advanced dashboards. Requests, failures, and one alert are enough.

10. Day 7 answer - Production polish

- Protect `main` and deploy prod only from `main`
- Add manual approval for prod if your GitHub plan supports it
- Write `DECISIONS.md` to explain why App Service, Static Web Apps, Key Vault, and Logic App were used

- Write `INTERVIEW\_NOTES.md` so the engineer can explain the project in 2 minutes
- Optional: install a self-hosted runner on the dev VM and run one small workflow on it
- Optional: upgrade to OIDC because it is more secure than long-lived client secrets

11. Interview-ready summary they should be able to say

"I built a startup-style Azure delivery platform for a .NET product using GitHub Actions, Bicep, App Service, Static Web Apps, Key Vault, managed identity, Application Insights, and a Logic App to automatically start a stopped VM. I used Cloud Shell for Azure work because the laptop was locked down, and I separated dev and prod with structured deployment controls."

12. References

- Azure Cloud Shell features: <https://learn.microsoft.com/en-us/azure/cloud-shell/features>
- GitHub environments: <https://docs.github.com/en/actions/reference/workflows-and-actions/deployments-and-environments>
- Deploy to App Service from GitHub Actions: <https://learn.microsoft.com/en-us/azure/app-service/deploy-github-actions>
- App Service Key Vault references: <https://learn.microsoft.com/en-us/azure/app-service/app-service-key-vault-references>
- Codeless Application Insights for App Service: <https://learn.microsoft.com/en-us/azure/azure-monitor/app/codeless-app-service>
- GitHub to Azure with OIDC: <https://learn.microsoft.com/en-us/azure/developer/github/connect-from-azure-openid-connect>